

Proving `exp100` in Capla

The plan, cut into tasks for separate agents (3 October 2026)

`code/APaul/exp100/exp100.c` computes an enclosure of $\exp(x)$ to about 156 bits on integers, and the filter `maybe_hard_bits`. This note plans its proof through Capla, as was done for `htr3` (`doc/htr3-capla-plan.typ`). The goal is one Rocq theorem:

if `maybe_hard_bits` returns 0 on the bits x_b of a double x , then x is not hard

with `hard` the predicate of `code/exptablekl/ExpHard.v`: $\exp(x)/v$ within 2^{-43} of an integer, $v = 2^{\max(e, -1022) - 53}$, e the binade of $\exp(x)$. The README of `exp100` gives the proof plan in six steps (word arithmetic, `exp_core` in integers, the constants, the real error bound, the decision, the assembly). Steps 3 to 5 have no C in them; this plan only changes steps 1, 2 and 6, which become proofs on Capla's semantics.

1 What was measured (the probe)

Everything in this section was run; the rest of the note is a plan.

- **The translation exists and agrees with the C.** `code/APaul/capla/exp100/exp100.b` (300 lines) is `exp100.c` in Capla. It compiles with Capla's `ccomp` (switch `capla`). `test_capla` runs it next to `exp100.c` on 10^6 inputs (a third random bit patterns, a third with $|x| < 1024$, a third uniform on $(-745.14, 709.79)$): **0 differences** in the return code, M , s and `maybe_hard_bits` (14 s for the whole run, both programs, `ccomp -02`).
- **The Rocq term builds.** `ccomp -dcaplarocq` gives `exp100.v` (6970 lines); it type-checks in 5.5 s. One trap: Capla's printer (`bfrontend/PrintCoq.ml`, line 193) writes `Salloc (i, e)` as a pair; the Makefile fixes it with `sed`. To report upstream.
- **One spec is proved.** `num_add_spec` (`proof/NumAddProof.v`, 138 lines, with `proof/ExpBase.v`, 145 lines: `val32`, `limbs`, the lemmas on `c & (2^32-1)` and `c >> 32`). It builds in 19 s. It took about one hour of agent time, set-up included; with the base in place, a function of the same kind (`num_zero`, `num_copy`, `num_sub`, `num_mul_small`) is GUESSED at 30 to 60 minutes.
- **The WP goals of a call with `alloc`, `free` and a returned value (`guess_n`)** are as expected: the scratch array is `repeat (defval u64) 7` (zeros), the call appears as an `eval_funcall` hypothesis, as in `SearchProof.v`.
- Not measured: the time of the WP on `exp_core` (about 1300 lines of the Rocq term) and on `decide`. This is the main open cost (see Risks).

2 The translation (`exp100.b`)

C (<code>exp100.c</code>)	Capla (<code>exp100.b</code>)
a number: <code>uint64_t a[NL]</code> , 6 limbs of 32 bits	<code>[u64; 6]</code> ; in Rocq a <code>list int64</code> of length 6, read by <code>val32</code> (limbs of 32 bits in words of 64) with <code>limbs</code> (every limb $< 2^{32}$)
<code>static const tables T[64][6], C[17][6], LN2, RMAX</code>	no globals in Capla: parameters <code>T: [[u64; 6]; 64]</code> , <code>C: [[u64; 6]; 17]</code> , <code>LN2</code> <code>RMAX: [u64; 6]</code> , filled by the C caller from <code>exp100_table.h</code> ; a row is passed as <code>T[j]</code> (a nested array is a <code>Varr</code> of <code>Varr</code> ; the call goes through <code>Sletref</code>)
<code>INV</code> , masks, in hexadecimal	decimal literals (no hexadecimal in Capla); <code>INV = 3098164009u64</code>

local scratch uint64_t p[2*NL]	let p = alloc u64, 12; ... free p; (zeros at start); in exp_core, scratch arrays come from the caller, since it returns early
for (i = NL-1; i >= 0; i--)	for k: u64 = 0 .. 6 { let i: u64 = 5 - k; ... }
shifts by an int	the shift amount must be u32 (or smaller): x >> (u32) k; Capla takes it modulo 64, C leaves ≥ 64 undefined
int for be, b, f	u64; the signed exponents hN, e, ve, fs are i64
int num_lt	-> bool
int64_t *s	s: mut [i64; 1] (no pointers to scalars)
maybe_hard_bits in one piece	exp_core, then decide(y, hN, ...); maybe_hard_bits only allocates, calls, frees
the union reading the bits of a double	unchanged: outside the proof, as in the C; the theorem is on x_i

What Capla cannot express, and the way around:

- **Constant tables.** Either (A) an `init_tables` function in Capla that writes the 498 limbs (as `init_coeffs` in `capla/poly.b`), or (B) parameters with a hypothesis on their contents. The plan takes (B): the hypothesis `tables_ok` says that the arrays are `map Int64.repr` of the lists of `code/APaul/exp100/rocq/ExpTable.v`, which `gen_v.py` makes from the same `exp100_table.h`. The C driver copies the header into the arrays; that copy is trusted (one `memcpy` per table). (A) is an optional later task (T14) that removes this trust at the price of a WP over 498 assignments.
- **Early returns with allocated arrays.** Every `alloc` needs its `free`, so functions that allocate have a single exit; `exp_core` and `decide`, which return early, allocate nothing.
- **No 128-bit type, no umulh needed:** the 32-bit limbs keep every product in a word, as in the C.

3 The statements

All in `int64` lists read by `val32`; every hypothesis “fits” is on $\mathbb{Z}$, never on `nat` (no $n < 2^{64}$ in `nat`). A spec has the shape of `addw_spec`: from `eval_funcall ge (Internal f) args e1 (Some result)`, the final arrays in `e1` (by `param k f`), their lengths, `limbs`, and the numbers. Below, a, b are the values `val32` of the inputs.

function	spec	loop invariant (step k)
<code>num_zero, num_copy</code>	$a' = 0; a' = b, \text{limbs}$	the first k limbs set, the others unchanged
<code>num_add (done)</code>	$a + b < 2^{192} \Rightarrow a' = a + b$	$\text{val}(a'_{<k}) + c2^{32k} = \text{val}(a_{<k}) + \text{val}(b_{<k}), c \leq 1$
<code>num_sub</code>	$a \geq b \Rightarrow a' = a - b$	$\text{val}(a'_{<k}) - c2^{32k} = \text{val}(a_{<k}) - \text{val}(b_{<k}), c \in \{0, 1\}$
<code>num_lt</code>	$\text{result} = (a <? b)$	the limbs 5, ..., 6 - k are equal; a <code>return</code> inside the loop: the post is on <code>Out_return</code>
<code>num_mul_small</code>	$w < 2^{32} \Rightarrow p' = aw$ (7 limbs)	as <code>num_add</code> , carry $< 2^{32}$
<code>num_mulshr</code>	$\lfloor ab/2^{160} \rfloor < 2^{192} \Rightarrow r' = \lfloor ab/2^{160} \rfloor$	outer: $\text{val}(p_{<i+6}) = \text{val}(a_{<i})b$, the rest 0; inner: the <code>mul_1</code> invariant; bound $a_i b_j + p + c \leq 2^{64} - 1$
<code>num_bitlen</code>	$\text{result} = \mathbb{Z}.\text{size } a$ (0 or $2^{b-1} \leq a < 2^b$)	b = size of the bits seen, by (i, k)

num_pow2, num_low	$f < 192 \Rightarrow a' = 2^f; a' = b \bmod 2^f$	num_low: limbs below $f/32$ copied, one masked, the rest 0
num_scale	$v < 2^{53}, \quad -2^{31} < e \leq 138 \Rightarrow a' = \lfloor v2^e \rfloor$	no loop; two cases
mul_ln2	$n < 2^{32} \Rightarrow q' = \lfloor n \text{ LN2} / 2^{32} \rfloor$	no loop
guess_n	the result is $< 2^{17}$ (its value is not needed: n is checked)	no loop
exp_core	the integer post below	Horner: $h = H_i$ after $16 - i$ steps
decide	result = 0 $\Rightarrow$ the integer facts of step 5 below	no loop
maybe_hard_bits	result = 0 $\Rightarrow$ exp_core returned 0 and decide returned 0	no loop

The integer post of exp_core (in ExpSpec.v, pure Rocq, on Z). With s = the sign bit of x_b , b_e its biased exponent, m its mantissa, when $b_e < 1033$:

- $X = \lfloor m'2^{b_e-915} \rfloor$ with m', b_e the mantissa and exponent with the implicit bit (subnormal case included), so $X = \lfloor |x| 2^{160} \rfloor$ for $x = \text{B2R}(\text{b64_of_bits } x_b)$ (a pure lemma, T9);
- $q(n) = \lfloor n \cdot \text{LN2} / 2^{32} \rfloor$; the result is 0 only if there is an $n < 2^{17}$ with $q(n) \leq X < q(n+1)$, and then $r = X - q(n)$ ($s = 0$) or $r = q(n+1) - X$ ($s = 1$), $r < \text{RMAX}$; $N_u = 2^{17} + n$ or $2^{17} - (n+1)$, $j = N_u \bmod 64$, $h_N = \lfloor N_u/64 \rfloor - 2048$;
- $H_{16} = C_{16}$, $H_i = \lfloor H_{i+1}r/2^{160} \rfloor + C_i$, and $y = \lfloor T_j H_0/2^{160} \rfloor$, all as functions on Z;
- y holds $y(\text{val32}, \text{limbs})$ and $\text{hs}[0]$ holds h_N .

No real number appears. The fit hypotheses of num_mulshr and num_add inside Horner ($H_i < 2^{192}$) are proved from $r < \text{RMAX}$ and the C_i by an induction on i on Z (T8), outside the WP.

The post of decide: if the result is 0 then, with $b = \text{Z.size } y$, $\text{Z.size } (y - 16) = \text{Z.size } (y + 16) = b$, $e = h_N - 160 + b - 1$, $f = \max(e, -1022) - 53 - (h_N - 160) \in [64, 184]$, and $2^{f-43} + 16 < \min(y \bmod 2^f, 2^f - y \bmod 2^f)$.

4 Capla proofs and pure Rocq

on Capla's semantics (switch capla, Rocq 9.0)	pure Rocq, no Capla
the specs of the table above; exp_core_spec and decide_spec only chain the specs of the callees; the final maybe_hard_bits theorem	ExpSpec.v: X, q, H_i, y as functions on Z; the bits of a double (Flocq); step 3, the constants (ExpConsts.v, the other agent); step 4, $ \exp(x) - y2^{h_N-160} \leq 16 \cdot 2^{h_N-160}$; step 5, the decision: the post of decide and step 4 give $\sim \text{hard } x$

The pure part is the same for the VST route if ExpSpec.v is stated on Z only (no int64, no val32). The plan is to write it once in code/APaul/exp100/rocq (switch native, Rocq 9.1, next to ExpConsts.v) and to build it also in the switch capla, as make_ports does for htr3; the switch capla has Interval 4.11.4, Bignums and Flocq 4.2.2. The bridge from val32 (Capla) to valZ (ExpConsts.v) is one lemma: $\text{val32 } (\text{map Int64.repr } l) = \text{valZ } l$ when every element is a limb.

The constants. ExpConsts.v (other agent) states T_ok, C_ok, LN2_ok, RMAX_ok on the lists of ExpTable.v. The Capla proof takes the tables as hypotheses tables_ok; T10 (step 4) uses the _ok lemmas; the final theorem keeps tables_ok as a hypothesis on the arguments (what the C driver passes).

5 The final theorem

```
Theorem maybe_hard_sound xb Ta Ca L2a RMa e1 res :
  tables_ok Ta Ca L2a RMa ->
  eval_funccall ge (Internal maybe_hard_bits) [Vint64 xb; Ta; Ca; L2a; RMa]
    e1 (Some res) ->
  res = Vint64 Int64.zero ->
  ~ hard (B2R (b64_of_bits xb)).
```

(`b64_of_bits` from Flocq's Bits; a NaN or an infinity never gives 0, since `exp_core` returns 1 for $b_e \geq 1033$.) The link with `ExpHard.v`: `hard`, `vof` and `binade` are used as they are; step 5 shows that b fixes `binade` (`exp x`) and that `vof x` = $2^{h_N - 160 + f}$, then that `exp(x)/vof(x)` is at distance more than 2^{-43} of every integer. It rests on `WP_sound` (admitted in Capla), on the C driver that fills the tables and reads the bits, and on the standard axioms of the reals.

6 Risks

- **The WP on `exp_core`.** About twenty calls, four early returns, and a loop, in one function: the WP goal is large, and `nia`, `lia` and `set` are slow or break in it (seen on `num_add`: a `nia` in the loop body ran for minutes; moved to a lemma outside, the proof takes 19 s). Plan: (1) all arithmetic in lemmas outside the WP context; (2) if `exp_core` is still too slow, cut it in Capla into `reduce` (up to r , N_u), `horner` and the final product, each with its own spec; this changes only the `.b` and keeps the test against the C.
- **`apply_WP_stmt` loses the return value:** copy the hypothesis `Some result = outcome_result_value` out with `have := first` (done in `SearchProof.v`); needed for `num_lt`, `num_bitlen`, `guess_n`, `exp_core`, `decide`, `maybe_hard_bits`.
- **Nested arrays and `Sletref`:** rows `T[j]`, `C[i]` go through `Sletref`, which no proof of `htr3` used. GUESSED risk: medium; T0 should prove one small call with a row before the others start.
- **`num_lt` returns inside a loop:** the invariant must also cover the `Out_return` outcome; no proof of `htr3` did this.
- **The printer bug on `Salloc`** (fixed by `sed`); other printer bugs may appear on constructs not yet seen.
- **Trust:** `WP_sound` is admitted, and the theorem is on Capla's semantics of the `.b`, not on `exp100.c`; the agreement of the two is tested (10^6 inputs), not proved.
- **Step 4** (the real error bound) is the largest pure piece and is shared with the VST route; its size does not depend on Capla.

7 The tasks

Each task is one file with fixed statements, written in T0 with **Admitted**; an agent replaces the **Admitted** by a proof and may use the statements of the other tasks, not their proofs. Estimates are GUESSED unless marked measured.

task	file	proves	needs, effort
T0	<code>exp100.b</code> , <code>ExpBase.v</code> , <code>ExpSpec.v</code> (statements), stubs	partly done: <code>.b</code> , test, Makefile, <code>ExpBase.v</code> . To do: the statements of every spec, <code>tables_ok</code> , the func- tions of <code>ExpSpec.v</code> , one row call (<code>T[j]</code>) through <code>Sletref</code>	–; 2 to 4 h
T1	<code>NumAddProof.v</code>	<code>num_add_spec:</code> done (measured: 138 lines, 1 h, 19 s)	T0

T2	NumBasicProof.v	num_zero, num_copy, num_pow2, num_low	T0; 2 to 4 h
T3	NumSubProof.v	num_sub_spec	T0; 1 h
T4	NumLtProof.v	num_lt_spec (return in the loop)	T0; 2 to 3 h
T5	NumMulProof.v	num_mul_small, num_mulshr (nested loops, alloc)	T0; 4 to 8 h
T6	NumBitsProof.v	num_bitlen, num_scale	T0; 3 to 5 h
T7	ReduceProof.v	mul_ln2, guess_n	T5; 1 to 2 h
T8	HornerZ.v (pure)	the bounds on Z: $X < 2^{170}$, $H_i < 2^{192}$, fits of every call of <code>exp_core</code> and <code>decide</code>	T0; 2 to 4 h
T9	DoubleBits.v (pure)	$X = \lfloor x 2^{160} \rfloor$ from the bits, with Flocq; sign and subnormals	T0; 3 to 6 h
T10	ExpError.v (pure, shared with VST)	step 4: $ \exp(x) - y2^{h_N-160} \leq 16 \cdot 2^{h_N-160}$ from the post of <code>exp_core</code> , <code>ExpConsts.v</code>	T0, constants; 1 to 3 days
T11	ExpDecide.v (pure, shared with VST)	step 5: the post of <code>decide</code> and step 4 give $\sim \text{hard } x$	T10 (statement); 4 to 8 h
T12	ExpCoreProof.v, DecideProof.v	exp_core_spec, decide_spec (chaining; the WP cost risk)	T2 to T8; 1 to 3 days
T13	ExpFinal.v	maybe_hard_sound, and the port of the pure files to the switch <code>capla</code>	T9, T11, T12; 3 to 6 h
T14 (optional)	InitTables.v	init_tables in Capla writes the tables; removes the trust in the C copy	T0; 4 h to 2 days

Order: T0 first (one agent, by hand). Then in parallel: T2, T3, T4, T5, T6, T8, T9, T10 (T10 can start at once: it needs only the statement of `ExpSpec.v` and the constants). Then T7 (after T5) and T11 (after T10's statement). Then T12. Then T13. T14 at any time after T0.

Total, GUESSED: 6 to 12 agent-days, of which T10 and T12 are more than half.

8 Capla or VST

The VST route (planned by another agent) proves `exp100.c` itself with VST's separation logic, as `code/APaul/vst/Verif_add.v`.

- **Trust base.** VST: VST's soundness is proved (on CompCert Clight); `clightgen` (the parser) is not; the tables are C globals inside the verified program, so nothing is assumed of a caller. Capla: `WP_sound` is admitted, the theorem is on a translation tested against the C (not proved equal), and the tables are trusted to be copied by the C driver (unless T14). On the trust base, VST is ahead.
- **Effort.** Capla: arrays are values (lists), no separation logic, no aliasing to reason about; the WP gives plain goals; one spec of a word loop cost one hour (measured). Its weak points are the size of the WP context (slow `nia`, `set`) and the few tactics. VST: mature `forward` tactics and `Verif_add.v`

to copy, but every array access carries a `data_at` and the 2-D tables need `field_address` lemmas; `semax_body` checks are slow on long functions too. GUESSED: the leaf functions cost about the same in both; `exp_core` is the larger unknown in both.

- **Shared work.** Steps 3, 4 and 5 (the constants, the error bound, the decision) are the same in both routes if stated on Z, and they are the larger part of the whole proof. Only steps 1, 2 and 6 differ.